

Simple Linear Regression

Concept, Mathematics, Worked Example, and Python Implementation

1. What is Simple Linear Regression?

Simple linear regression is a supervised machine learning technique used to model the relationship between a single independent variable (predictor, X) and a continuous dependent variable (target, Y). It assumes the relationship between X and Y can be approximated by a straight line. The goal is to find the line that best fits the observed data by minimizing the difference between actual and predicted values.

2. The Mathematical Model

The relationship is expressed using the equation of a straight line:

$$\hat{Y} = \beta_0 + \beta_1 X + \epsilon$$

Where $\hat{Y}$ is the predicted (dependent) variable, X is the independent variable, β_0 (intercept) is the value of $\hat{Y}$ when X is zero, β_1 (slope/coefficient) represents the change in $\hat{Y}$ for a one-unit change in X , and ϵ is the random error term that accounts for variation not explained by the model.

How the Line is Fit: Ordinary Least Squares (OLS)

The best-fit line is found by minimizing the Sum of Squared Errors (SSE) between the actual values and the predicted values on the line:

$$SSE = \sum (y_i - \hat{y}_i)^2$$

This approach, called Ordinary Least Squares, guarantees a unique solution for the slope and intercept that minimizes total prediction error across all data points.

3. Worked Example

Suppose we want to predict a house's **price (in lakh PKR)** based on its **size (in marlas)**. Consider the following small dataset:

Size (marlas) — X	Price (lakh PKR) — Y
3	45
5	68
7	90
10	130
12	155

Fitting a simple linear regression model to this data (using the least squares method) produces approximately:

$$\hat{Y} = 15.8 + 11.4 \times X$$

This means the base price is roughly **15.8 lakh PKR** (intercept), and each additional **marla** adds about **11.4 lakh PKR** (slope) to the predicted price. For a 8-marla house, the predicted price would be: ■ = $15.8 + 11.4 \times 8 = 106.8$ lakh PKR.

4. Python Implementation (scikit-learn)

Below is a complete, minimal implementation of simple linear regression using pandas and scikit-learn, following the same workflow used in a real project (feature/target split, train-test split, model fitting, and evaluation):

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv('data.csv')

# Define feature (X) and target (y)
X = df[['radius_mean']].values
y = df['area_mean'].values

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Evaluate performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Slope (coefficient): {model.coef_[0]:.4f}")
print(f"Intercept: {model.intercept_:.4f}")
print(f"MSE: {mse:.2f}")
print(f"R-squared: {r2:.4f}")
```

Making a New Prediction

```
# Predict area for a new radius value
new_radius = [[15]]
predicted_area = model.predict(new_radius)
print(f"Predicted area: {predicted_area[0]:.2f}")
```

5. Evaluating Model Performance

- **Mean Squared Error (MSE):** Average of squared differences between actual and predicted values. Lower is better; heavily penalizes large errors.
- **R-squared (R²):** Proportion of variance in Y explained by X, ranging from 0 to 1. An R² of 0.85 means 85% of the variability in the target is explained by the model.

- **Residuals:** The difference between actual and predicted values ($y - \hat{y}$). Residuals should be randomly scattered around zero with no visible pattern, indicating the linear model is appropriate.

6. Key Assumptions

- Linearity: the relationship between X and Y is approximately linear.
- Independence: observations are independent of one another.
- Homoscedasticity: residuals have constant variance across all values of X.
- Normality: residuals are approximately normally distributed (checked via a Q-Q plot).

7. Common Applications

- Predicting house prices from square footage or plot size
- Estimating sales revenue from advertising spend
- Forecasting exam scores based on hours studied
- Modeling relationships between physical measurements (e.g., radius vs. area, height vs. weight)